

Chapter 2

Advanced HTML Content Structure & HTML5 Semantics

Theory & Concepts

This chapter will deepen students' understanding of how to structure more complex HTML content, specifically for presenting data in tables and utilizing HTML5 semantic elements for report readability and accessibility.

2.1 HTML Lists:

- **Unordered Lists (and):** Used for lists of items without a specific order (e.g., chart of accounts, list of fixed assets).
- **Ordered Lists (and):** Used for lists of items with a specific order (e.g., steps in the accounting cycle, audit phases).
- type attribute on (e.g., type="A", type="i") to change the numbering style.
- *Nested Lists:* Creating lists within lists for hierarchical information (e.g., parent accounts and sub-accounts).

2.2 HTML Tables (For Structured Data Presentation):

- **Purpose of Tables:** Ideal for displaying data in rows and columns (e.g., financial reports, transaction lists, asset lists).
- Basic Table Structure:
 - <table>: The table wrapper element.
 - <thead>: The table *header* section (column titles).
 - <tbody>: The table *body* section (main data).
 - <tfoot>: The table *footer* section (e.g., totals or footnotes).
 - <tr>: *Table Row*.
 - <th>: *Table Header* cell (usually bold and centered).
 - <td>: *Table Data* cell.
- Merging Cells:
 - colspan: Merges multiple columns into one cell. Important for table titles spanning multiple data columns (e.g., "Accounting Period" above "Debit" and "Credit").
 - rowspan: Merges multiple rows into one cell. Useful for grouping data with the same label across several rows.
- Importance of the scope attribute on <th> (e.g., scope="col", scope="row") for accessibility.

2.3 HTML5 Semantics (Importance of Structural Meaning):

- **What is Semantic HTML?** The use of HTML elements that carry meaning corresponding to their content, not just their appearance.

- Why Semantics are Important:
 - **Accessibility:** Helps *screen readers* understand page structure for users with disabilities.
 - **SEO (Search Engine Optimization):** Makes it easier for *search engines* to index and understand page content.
 - **Code Readability:** Makes code easier for other developers to understand.
 - **Maintainability:** Facilitates future maintenance and development.
- Key Semantic Elements for Report/Application Layout:
 - `<header>`: The introductory part of a page or section (e.g., report title, logo).
 - `<nav>`: Navigation section (menu links).
 - `<main>`: The main and unique content of the document.
 - `<article>`: Self-contained content that can be independently distributed (e.g., a news article, a blog post, or a standalone report section).
 - `<section>`: A thematic grouping of content, usually with its own heading (e.g., "Executive Summary", "Revenue Analysis").
 - `<aside>`: Content indirectly related to the main content (e.g., a *sidebar* with ads, a list of related links).
 - `<footer>`: The closing part of a page or section (e.g., copyright information, brief contact).
- Other Semantic Elements: `<figure>`, `<figcaption>`, `<time>`.

2.4 Git & GitHub (Advanced Basics):

- **Git Status (Detailed):** Understanding git status output to track modified, new, or deleted files.
- **Good Commit Messages:** The importance of descriptive and concise *commit messages* for easy project history tracing.
- **Introduction to GitHub:** A *Git repository hosting platform*. Basic functions (viewing *commits*, *branches*).
- **Connecting Local Git to GitHub:** `git remote add origin <URL_repo>` and `git push -u origin main`.

Practical Exercises

This practical session will focus on building financial data structures using tables and lists, and applying HTML5 semantic elements to create well-organized web reports.

Class Exercises

1. **Exercise 2.1: Creating Hierarchical Account Lists & Accounting Procedure Lists.**
 - Inside your project's root folder `week_2/`, create a new HTML file named `account_procedure_lists.html`.
 - **Hierarchical Account List:** Create a fictional chart of accounts with a hierarchy using *nested lists*. Example:

```
<h1>Chart of Accounts - Maju Jaya Company</h1>
<h2>By Account Code</h2>
<ol>
  <li>1000 - Assets
    <ul>
      <li>1100 - Current Assets
        <ol>
          <li>1110 - Cash</li>
          <li>1120 - Bank</li>
          <li>1130 - Accounts Receivable</li>
          <li>1140 - Inventory</li>
        </ol>
      </li>
      <li>1200 - Fixed Assets
        <ol>
          <li>1210 - Land</li>
          <li>1220 - Buildings</li>
          <li>1230 - Office Equipment</li>
        </ol>
      </li>
    </ul>
  </li>
  <li>2000 - Liabilities
    <ul>
      <li>2100 - Current Liabilities
        <ol>
          <li>2110 - Accounts Payable</li>
          <li>2120 - Salaries Payable</li>
        </ol>
      </li>
      <li>2200 - Long-Term Liabilities
        <ol>
          <li>2210 - Bank Loans</li>
        </ol>
      </li>
    </ul>
  </li>
  <li>3000 - Equity
    <ul>
      <li>3100 - Share Capital</li>
      <li>3110 - Retained Earnings</li>
    </ul>
  </li>
  <li>4000 - Revenue
    <ul>
      <li>4100 - Service Revenue</li>
    </ul>
  </li>
  <li>5000 - Expenses
    <ul>
      <li>5100 - Salaries Expense</li>
      <li>5200 - Rent Expense</li>
    </ul>
  </li>
</ol>
```

- **Simple Accounting Procedure List:** At the bottom of the same page, create a list of steps for a basic accounting procedure (e.g., "Cash Receipt Recording Procedure").

`<h2>Cash Receipt Recording Procedure</h2>`

`<ol type="1">`

`<li>Receive transaction evidence (e.g., receipt).</li>`

`<li>Verify completeness and validity of transaction evidence.</li>`

`<li>Identify involved accounts (e.g., Cash, Accounts Receivable/Revenue).</li>`

`<li>Record transaction in the Cash Receipts Journal.</li>`

`<li>Post to the General Ledger.</li>`

`<li>Archive transaction evidence.</li>`

`</ol>`

- Save and view the results in the *browser*. Observe how the lists are automatically indented and the use of the type attribute on the ordered list.

2. Exercise 2.2: Creating an Advanced General Journal Table & Cash Movement Report.

- Inside the week_2/ folder, create a new HTML file named transaction_reports_exercise.html.
- **Advanced General Journal Table:** Create a general journal table with `<thead>`, `<tbody>`, and `<tfoot>`. This table will have columns for Date, Ref. No., Description, Account, Debit (IDR), and Credit (IDR).
- Fill in at least 5 rows of dummy transactions covering different accounts, ensuring total debits equal total credits.
- **Using colspan and rowspan:**
 - Use colspan for the "Transaction Details" title spanning the Date, Ref. No., and Description columns.
 - Use rowspan to group transaction rows if there are compound transactions (more than one debit/credit account for a single date/reference number).
 - Include a `<tfoot>` row for "TOTAL Debit" and "TOTAL Credit".

```
<h1>General Journal</h1>
<table>
  <caption>Transactions for March 2025</caption>
  <thead>
    <tr>
      <th colspan="3">Transaction Details</th>
      <th>Account</th>
      <th>Debit (IDR)</th>
      <th>Credit (IDR)</th>
    </tr>
    <tr>
      <th>Date</th>
      <th>Ref. No.</th>
      <th>Description</th>
```

```

        <th></th> <!-- Empty Account column for header alignment -
->
        <th></th> <!-- Empty Debit column for header alignment -->
        <th></th> <!-- Empty Credit column for header alignment --
>
    </tr>
</thead>
<tbody>
    <tr>
        <td>2025-03-01</td>
        <td>TRX001</td>
        <td>Cash Receipt from Service Revenue</td>
        <td>Cash</td>
        <td>10,000,000</td>
        <td></td>
    </tr>
    <tr>
        <td colspan="3"></td> <!-- Merging transaction detail
cells -->
        <td>    Service Revenue</td>
        <td></td>
        <td>10,000,000</td>
    </tr>
    <tr>
        <td rowspan="2">2025-03-05</td> <!-- Merging Date row -->
        <td rowspan="2">TRX002</td> <!-- Merging Ref. No. row -->
        <td rowspan="2">Cash Purchase of Office Equipment</td> <!--
- Merging Description row -->
        <td>Office Equipment</td>
        <td>2,500,000</td>
        <td></td>
    </tr>
    <tr>
        <td>    Cash</td>
        <td></td>
        <td>2,500,000</td>
    </tr>
    <!-- Add 2-3 more transactions with rowspan/colspan if
relevant -->
    <tr>
        <td>2025-03-10</td>
        <td>TRX003</td>
        <td>Payment of Employee Salaries</td>
        <td>Salaries Expense</td>
        <td>1,500,000</td>
        <td></td>
    </tr>
    <tr>
        <td colspan="3"></td>
        <td>    Cash</td>
        <td></td>
        <td>1,500,000</td>
    </tr>
</tbody>
<tfoot>
    <tr>
        <td colspan="4"><strong>TOTAL</strong></td>

```

```
 <strong>14,000,000</strong></td> <!-- Adjust to your Debit total -->  <strong>14,000,000</strong></td> <!-- Adjust to your Credit total --> </tr> </tfoot> </table> | |
```

- **Simple Cash Movement Report (Separate Table):** Below the general journal table, create a separate table for cash movement. Columns: Date, Description, Receipts, Disbursements, Balance.

```

<h2>Cash Movement Report (Summary)</h2>
<table>
  <thead>
    <tr>
      <th>Date</th>
      <th>Description</th>
      <th>Receipts (IDR)</th>
      <th>Disbursements (IDR)</th>
      <th>Balance (IDR)</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>2025-03-01</td>
      <td>Beginning Balance</td>
      <td></td>
      <td></td>
      <td>5,000,000</td>
    </tr>
    <tr>
      <td>2025-03-01</td>
      <td>Receipt of Accounts Receivable</td>
      <td>5,000,000</td>
      <td></td>
      <td>10,000,000</td> <!-- 5M + 5M -->
    </tr>
    <tr>
      <td>2025-03-05</td>
      <td>Equipment Purchase</td>
      <td></td>
      <td>2,500,000</td>
      <td>7,500,000</td> <!-- 10M - 2.5M -->
    </tr>
    <tr>
      <td>2025-03-10</td>
      <td>Salary Payment</td>
      <td></td>
      <td>1,500,000</td>
      <td>6,000,000</td> <!-- 7.5M - 1.5M -->
    </tr>
  </tbody>
</table>

```

```

        <td colspan="4"><strong>Ending Balance</strong></td>
        <td><strong>6,000,000</strong></td>
    </tr>
</tfoot>
</table>

```

- Save and observe the display of both tables in the *browser*.
3. **Exercise 2.3: Refactoring tax_consultant_profile.html from Chapter 1 with HTML5 Semantics.**
- Open the file week_1/tax_consultant_profile/index.html from Assignment 1.1 in Chapter 1.
 - **Copy its entire content** to a new file named week_2/tax_consultant_profile_semantic/index.html. Also, copy the assets/images/ folder into tax_consultant_profile_semantic/.
 - Refactor the structure of index.html using appropriate HTML5 semantic elements:
 - Wrap the main title and logo within a <header> tag.
 - Create a simple <nav> element after the <header>, containing dummy links like "Home" and "Services".
 - Wrap all main content (About Us, Contact Information, Our Main Services) within a <main> tag.
 - For each section like "About Us", "Contact Information", and "Our Main Services", wrap them individually in <section> tags. Also give them appropriate id attributes (e.g., <section id="about-us">).
 - If there's content that could be considered self-contained (e.g., "Client Testimonials" if you added them), it can be wrapped in an <article> tag.
 - If there's supplementary information (e.g., "Our Awards"), it can be placed in an <aside> tag.
 - Add a <footer> for copyright information and brief contact details at the very bottom of the page.
 - Verify that the page still displays the same in the *browser* even though its structure has been changed to be more semantic. Use *Developer Tools* (Inspect Element) to observe the new HTML element structure.
4. **Exercise Git 2.4: Committing Class Exercise Changes.**
- Open the terminal in your project's root folder.
 - Run the command:


```
git add week_2/
git commit -m "Class Exercise: Advanced lists & tables, semantic refactoring of tax consultant profile"
```
 - Use git status and git log to verify.

Individual Practical Assignment

Students will complete this assignment individually. This assignment will test their ability to create financial data reports using complex tables and HTML5 semantic structures, and integrate them back into the context of the company from Chapter 1.

Assignment 2.1: Web Page "Statement of Financial Position (Balance Sheet) of [Your Company Name from Chapter 1] & Summary of Changes in Equity"

- Create a new folder named `financial_statements_assignment/` inside the `week_2/` folder.
- Inside it, create a new HTML file named `index.html`.
- Fill in the complete HTML structure with relevant `<meta>` tags and an appropriate `<title>` (e.g., Statement of Financial Position of [Your Company Name]).
- **Task:** Create a *web page* presenting a simple Statement of Financial Position (Balance Sheet) and Summary of Changes in Equity for your fictional company from Chapter 1 (tax_consultant_profile).
- **Mandatory Requirements (All must be included):**
 - Use the semantic element `<header>` for the report title (e.g., "STATEMENT OF FINANCIAL POSITION", "PT. [Your Company Name]", "As of December 31, [Fictional Year]").
 - Use the `<main>` element to wrap all primary report content.
 - Inside `<main>`, create a `<section>` with the title "Statement of Financial Position". Within it, create a `<table>` representing the Balance Sheet:
 - The table must have `<thead>` (column titles), `<tbody>` (main data), and `<tfoot>` (for totals or notes).
 - Required columns: Account, Amount (IDR).
 - Include at least (with reasonable fictional values):
 - 5 Asset Accounts (at least 3 current, 2 fixed).
 - 3 Liability Accounts (at least 2 current, 1 long-term).
 - 3 Equity Accounts (e.g., Share Capital, Retained Earnings/Accumulated Profit, Drawings/Dividends).
 - **Apply `colspan`** for sub-section titles such as "ASSETS", "LIABILITIES", "EQUITY" or for "Total Assets", "Total Liabilities & Equity" to make the table more structured.
 - **Apply `rowspan`** if there is a need to group similar rows (e.g., one "Equipment" row could cover "Cost" and "Accumulated Depreciation" in subsequent rows, then `rowspan` for the "Equipment" header). This is optional if the table is simple.
 - Ensure the table has rows for "TOTAL ASSETS" and "TOTAL LIABILITIES & EQUITY" with equal values (to show the balance of the balance sheet).
 - Below the balance sheet table, create a new `<section>` with the title "Summary of Changes in Equity". Within it, create an **Ordered List (`<ol>`)** explaining 3-5 points

of equity changes during the period (e.g., "Beginning equity balance", "Capital contributions from investors", "Net profit for the period", "Dividend withdrawals/drawings", "Ending equity balance").

- Add a **<footer>** with copyright information or a brief footnote at the bottom of the page (e.g., "Created by [Your Name] - [Your NIM] for Basic Programming course").
- Ensure all elements used have content relevant to the theme of the Statement of Financial Position and Summary of Changes in Equity.
- **Save Git Changes:** After all HTML files are ready, open the terminal in your project's root folder (basic_programming_ais_YourNIM/) and run the commands:
git add week_2/financial_statements_assignment/
git commit -m "Completed Assignment 2.1: Statement of Financial Position and Changes in Equity"

Submission Method

Students will create a **PDF document** containing:

1. **HTML Code** from week_2/financial_statements_assignment/index.html (Assignment 2.1). Ensure the code is neat, properly indented, and easy to read.
2. **Screenshots of the index.html page in the browser.** Ensure the balance sheet table is clearly visible with its *header*, *body*, *footer*, and proper use of *colspan* and *rowspan*. Also include a screenshot of the equity changes list.
3. **Screenshot of the terminal/CMD** showing the output of the git log command after the last commit for Assignment 2.1. This is to verify that the commit has been made.
4. The PDF should be printed and submitted to the lecturer for signature.